



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

КБ-3 «Разработка программных решений и системное программирование»

Отчет по практической работе №3

«Тестирование маршрутов до бэкенд ресурсов»

по дисциплине

«Сборка, тестирование и верификация программного продукта»

Выполнил студент группы БСБО-09-23

Фёдоров Антон Сергеевич

Работа выполнена

«01» марта 2026 г.

Москва, 2026 г.

СОДЕРЖАНИЕ

1. ЦЕЛЬ И ПЛАН РАБОТЫ	3
2. ОПИСАНИЕ РЕАЛИЗОВАННЫХ МАРШРУТОВ И СТРАНИЦ.....	4
3. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ	5
4. ИСХОДНЫЙ КОД	7
4.1. Настройка маршрутизации.....	7
4.2. Функции-обработчики	7
4.3. HTML-шаблоны.....	8
4.4. Модуль тестирования.....	10
ВЫВОДЫ	11

1. ЦЕЛЬ И ПЛАН РАБОТЫ

Цель: расширить сферу применения инструментов модульного тестирования в части тестирования маршрутов до бэкенд ресурсов.

План:

- Изучить формирование адресов для доступа к веб-ресурсов (http, url).
- Изучить маршрутизацию фреймворка Django.
- Познакомиться с использованием HTML шаблонов на стороне бэкенда.
- Изучить требования к заданию.
- Выполнить практическое задание согласно требованиям.
- Оформить отчёт.

2. ОПИСАНИЕ РЕАЛИЗОВАННЫХ МАРШРУТОВ И СТРАНИЦ

Для проверки работоспособности системы маршрутизации было создано веб-приложение с каталогом аккумуляторов. Были реализованы следующие страницы и соответствующие им URL-адреса:

- Главная страница (список товаров).

- URL: /

- Описание: отображает список всех доступных аккумуляторов с их ценами. Каждое название товара является ссылкой на страницу с детальным описанием. Реализована функцией `product_list` в файле `views.py`.

2. Страница детального описания товара.

- URL: `/product/<int:pk>/`

- Описание: отображает подробную информацию о конкретном аккумуляторе (название, характеристики, цена). Идентификатор товара `pk` передаётся в URL как целое число, что реализуется с помощью конвертера пути `int:pk`. Обработчик — функция `product_detail`.

3. Страница «О магазине».

- URL: `/about/`

- Описание: Статическая страница с общей информацией о сервисе. Реализована функцией `about`.

3. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Для проверки корректности настройки маршрутов использовался встроенный в Django фреймворк для модульного тестирования на базе стандартной библиотеки Python unittest. Были написаны тесты, проверяющие, что каждый из трёх реализованных URL-адресов возвращает код ответа 200 ОК, что свидетельствует об их доступности.

Тестирование прошло успешно, все тесты выполнены без ошибок, что подтверждает корректную работу системы маршрутизации.

```
ariko@MacBook-Air-Anton catalog_project % python3 manage.py test
Found 3 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
...
-----
Ran 3 tests in 0.012s

OK
Destroying test database for alias 'default'...
```

Рисунок 1. Результат выполнения модульных тестов

Ниже представлены скриншоты работающего веб-приложения.

Магазин аккумуляторов

[О магазине](#)

- [Автомобильный аккумулятор VARTA](#) - 6500 руб.
- [Power Bank Xiaomi 20000mAh](#) - 2500 руб.
- [Аккумуляторные батарейки AA](#) - 1200 руб.

Рисунок 2. Главная страница со списком аккумуляторов

[Назад к списку аккумуляторов](#)

Автомобильный аккумулятор VARTA

Характеристики: Емкость 60 Ач, пусковой ток 540 А, обратная полярность.

Цена: 6500 руб.

Рисунок 3. Страница детального описания товара

О нашем магазине

Мы специализируемся на продаже надежных автомобильных, портативных и бытовых аккумуляторов для любых ваших нужд.

Рисунок 4. Страница «О магазине»

4. ИСХОДНЫЙ КОД

4.1. Настройка маршрутизации

Маршрутизация в проекте разделена на два уровня. Главный файл `catalog_project/urls.py` делегирует обработку всех адресов приложению `catalog`.

Листинг 1. Файл `catalog_project/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('catalog.urls')),
]
```

Файл `catalog/urls.py` содержит конкретные шаблоны адресов и связывает их с функциями-обработчиками из `views.py`.

Листинг 2. Файл `catalog/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.product_list, name='product_list'),
    path('product/<int:pk>/', views.product_detail,
         name='product_detail'),
    path('about/', views.about, name='about'),
]
```

4.2. Функции-обработчики

В файле `catalog/views.py` определены функции, которые принимают HTTP-запрос и возвращают HTTP-ответ, отрисовывая соответствующий HTML-шаблон. Для имитации базы данных используется статический список словарей `products_data`.

Листинг 3. Файл `catalog/views.py`

```
from django.shortcuts import render

products_data = [
```

```

        {'id': 1, 'name': 'Автомобильный аккумулятор VARTA', 'desc':
'Емкость 60 Ач, пусковой ток 540 А, обратная полярность.', 'price':
6500},
        {'id': 2, 'name': 'Power Bank Xiaomi 20000mAh', 'desc':
'Портативный внешний аккумулятор с поддержкой быстрой зарядки.',
'price': 2500},
        {'id': 3, 'name': 'Аккумуляторные батарейки AA', 'desc': 'Комплект
Ni-MH аккумуляторов, 4 штуки, емкость 2500 мАч.', 'price': 1200},
    ]

def product_list(request):
    return render(request, 'catalog/product_list.html', {'products':
products_data})

def product_detail(request, pk):
    product = next((item for item in products_data if item['id'] ==
pk), None)
    return render(request, 'catalog/product_detail.html', {'product':
product})

def about(request):
    return render(request, 'catalog/about.html')

```

4.3. HTML-шаблоны

Для отображения страниц используются три HTML-шаблона.

Листинг 4. Шаблон product_list.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Каталог аккумуляторов</title>
</head>
<body>
    <h1>Магазин аккумуляторов</h1>
    <a href="{% url 'about' %}">О магазине</a>
    <hr>
    <ul>
        {% for product in products %}

```



```

        <li>
            <a href="{% url 'product_detail' product.id %}">{{
product.name }}</a> - {{ product.price }} руб.
        </li>
    {% endfor %}
</ul>
</body>
</html>

```

Листинг 5. Шаблон product_detail.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Информация об аккумуляторе</title>
</head>
<body>
    <a href="{% url 'product_list' %}">Назад к списку
аккумуляторов</a>
    <hr>
    {% if product %}
        <h1>{{ product.name }}</h1>
        <p>Характеристики: {{ product.desc }}</p>
        <p>Цена: {{ product.price }} руб.</p>
    {% else %}
        <h1>Аккумулятор не найден</h1>
    {% endif %}
</body>
</html>

```

Листинг 6. Шаблон about.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>0 магазине аккумуляторов</title>
</head>
<body>
    <a href="{% url 'product_list' %}">Назад к списку
аккумуляторов</a>

```

```
<hr>
<h1>О нашем магазине</h1>
<p>Мы специализируемся на продаже надежных автомобильных,
портативных и бытовых аккумуляторов для любых ваших нужд.</p>
</body>
</html>
```

4.4. Модуль тестирования

Архитектура тестов построена на базе стандартного класса `django.test.TestCase`. В модуле реализованы три тестовых метода, каждый из которых проверяет доступность одного из маршрутов приложения.

Листинг 7. Исходный код `catalog/tests.py`

```
from django.test import TestCase
from django.urls import reverse

class CatalogRoutesTest(TestCase):
    def test_product_list_route(self):
        response = self.client.get(reverse('product_list'))
        self.assertEqual(response.status_code, 200)

    def test_product_detail_route(self):
        # Проверяем, что страница аккумулятора с id=1 загружается
        # успешно (код 200)
        response = self.client.get(reverse('product_detail',
        args=[1]))
        self.assertEqual(response.status_code, 200)

    def test_about_route(self):
        response = self.client.get(reverse('about'))
        self.assertEqual(response.status_code, 200)
```

ВЫВОДЫ

В результате выполнения практической работы было создано простое веб-приложение на базе фреймворка Django для демонстрации каталога товаров.

В ходе работы была изучена и настроена система маршрутизации Django, включая использование динамических сегментов URL с конвертером пути (<int:pk>) для создания страниц с детальным описанием объектов. Были созданы функции-обработчики, которые получают данные и передают их в HTML-шаблоны для отрисовки.

Для проверки корректности работы реализованных маршрутов был написан набор автоматизированных модульных тестов с использованием встроенных инструментов Django. Тесты успешно пройдены, что подтверждает правильность настройки URL-адресов приложения. Таким образом, цель практической работы полностью достигнута.